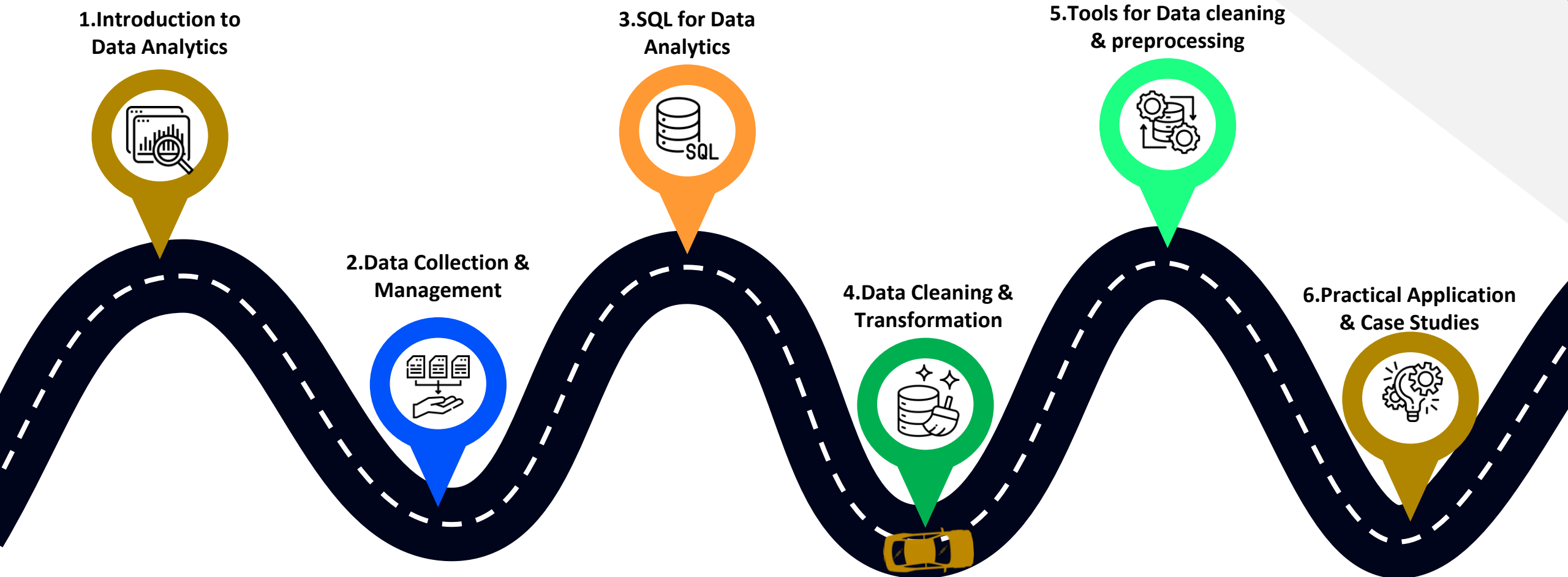


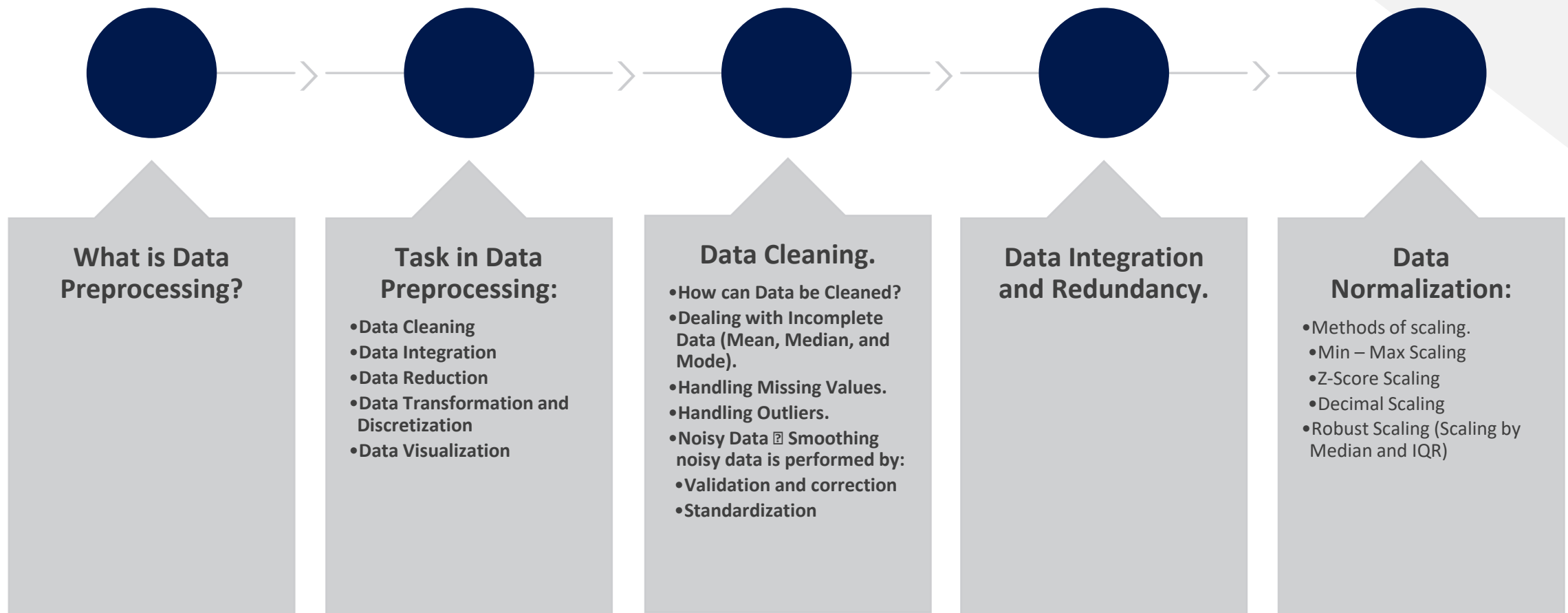


Part Five: Data Cleaning, and Transformation

Course Road Map



Agenda:



Agenda:

What is Data Preprocessing?

Task in Data Preprocessing:

- Data Cleaning
- Data Integration
- Data Reduction
- Data Transformation and Discretization
- Data Visualization

Data Cleaning.

- How can Data be Cleaned?
- Dealing with Incomplete Data (Mean, Median, and Mode).
- Handling Missing Values.
- Handling Outliers.
- Noisy Data ? Smoothing noisy data is performed by:
 - Validation and correction
 - Standardization

Agenda cont..:

Data Integration and Redundancy.

Data Normalization:

- Methods of scaling.
 - Min – Max Scaling
 - Z-Score Scaling
 - Decimal Scaling
- Robust Scaling (Scaling by Median and IQR)

Data Visualization:

- Intro to Visualization.
- Different visualization types using Matplotlib, Seaborn, and Plotly.

Data Preprocessing



What is Data Preprocessing

- The Data preprocessing is a crucial step in the data analysis and machine learning pipeline.
- It involves preparing raw data for further analysis and modelling and is an important factor in the success of any data project.



Task in Data Preprocessing

- ❑ **Data Cleaning:**

Fill in missing values, smooth noisy data, identify or remove outliers, and resolve inconsistencies.

- ❑ **Data Integration:**

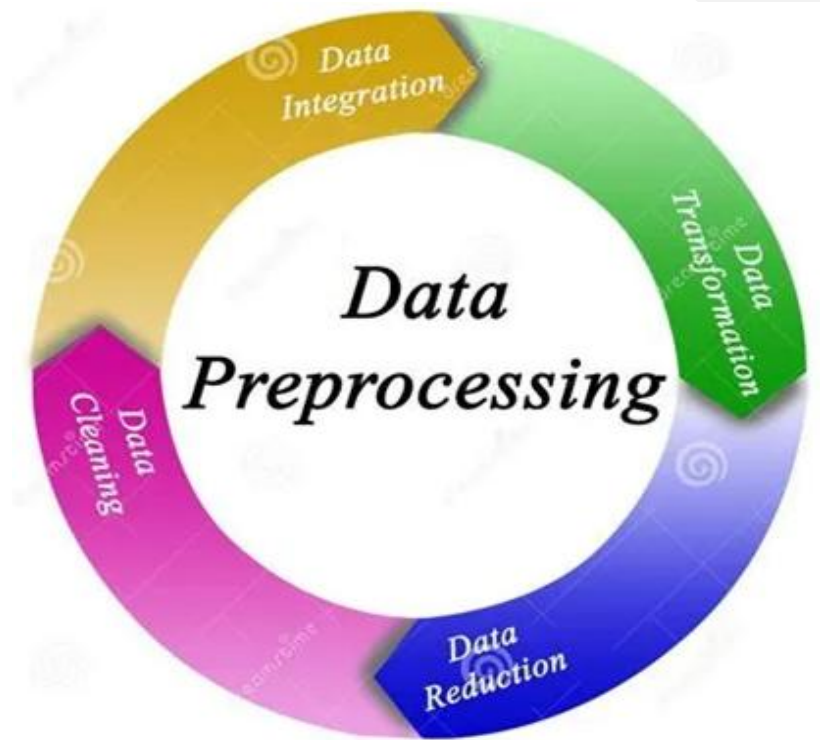
using multiple databases, data cubes, or files.

- ❑ **Data Reduction:**

reducing the volume but producing the same or similar analytical results.

- ❑ **Data Transformation and Discretization**

normalization and aggregation.





NTI
Data Analysis

DATA CLEANING

Data Cleaning

Data cleaning is the process of removing *incorrect data*, *incomplete data*, and *inaccurate data* from the datasets, and it also replaces the missing values. The data to be analyzed may be:

- **Incomplete:** where the data is missing (empty record).
- **Noisy:** where data may contain errors (data that have incorrect values -Gender Z)or outlier values (Outliers are data values that deviate from expected values of the rest of the data set) (Age 2500)
- **Inconsistent:** where data may contain discrepancies in the values (age – birthdate) , (Total-price and (unit-price and quantity).

Importance of Data Cleaning:

- **Garbage In, Garbage Out:** Flawed data leads to unreliable results. Clean data ensures accurate insights
- **Improved Decision Making:** Clean data empowers you to make informed and data-driven decisions.
- **Enhanced Analysis:** Accurate data facilitates efficient and effective data analysis.
- **Accurate Visualizations:** Clean data fosters the creation of trustworthy and impactful data visualizations.



How can Data be Cleaned?

Filling-in Missing Values:

- Smoothing Noisy Data (Validation and correction – Standardization)
- Identifying and Removing Outliers Using Inter-quartile Range
- Resolving Inconsistency (age – birthdate)

Typical Example

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	−40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Sample Table

Typical Example (Incomplete Data)

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	-40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Sample Table

Typical Example (Data Values Errors)

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	-40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Sample Table

Reasons:

- The Zip code consists of five digits and cannot contain any letters
- Income must be positive number
- Age must be positive number

Typical Example (Outlier Values)

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	-40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Sample Table

Reasons:

- Outliers are data values that deviate from expected values of the rest of the data set
- The values 10000000 and -40000 look very divergent from the rest of values

Typical Example (Ambiguity)

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	−40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Sample Table

Reasons:

- “S” in Marital Status could refer to “Single” or “Separated”
- So, there is a kind of **ambiguity** in the data

Reasons for Incomplete Data

- **Relevant data may not be recorded because:**
 - A misunderstanding from the data entry persons
 - Equipment failure
- **Relevant data may not be available because it is unknown or providing it is optional**

Dealing with Incomplete Data

There are several ways to deal with missing data:

- Replace the missing value with some default value
- Replace the missing value with the field mean for the fields that take numerical values or the mode (if exists) for the fields that take categorical values
- Replace the missing values with a value generated at random from the field distribution observed

Mean, Median, and Mode

- The mean for a population of size n can be computed by:

$$\bar{x} = \frac{1}{n} \cdot \sum_{i=1}^n x_i$$

- Consider the following list of 9 numbers:

13, 15, 12, 17, 22, 11, 13, 19, 12

Mean = $(13 + 15 + 12 + 17 + 22 + 11 + 13 + 19 + 12)/9 = 14.88889$

Mean, Median, and Mode Cont..

The median is the middle value of the ordered list of numbers.

- Consider the following list of 9 numbers:

13, 15, 12, 17, 22, 11, 13, 19, 12

- To compute the median, you need first to order the numbers:

11, 12, 12, 13, 13, 15, 17, 19, 22

Hence, the median is **13**

Mean, Median, and Mode Cont..

The median is the middle value of the ordered list of numbers.

- Consider the following list of 10 numbers:

13, 15, 12, 17, 22, 11, 13, 19, 12, 14

- To compute the median, you need first to order the numbers:

11, 12, 12, 13, 13, 14, 15, 17, 19, 22

Hence, the median is $(13 + 14)/2 = 13.5$

Mean, Median, and Mode Cont..

The mode of a set of data is the value in the set that occurs most often.

□ Consider the following list of numbers:

13, 15, 12, 17, 22, 11, 13, 19, 13

Number	Occurrence	Number	Occurrence
13	3	22	1
15	1	11	1
12	1	19	1
17	1		

Mode is 13

Mean, Median, and Mode Cont..

The mode of a set of data is the value in the set that occurs most often.

□ Consider the following list of numbers:

13, 15, 12, 17, 22, 11, 13, 19, 12

Number	Occurrence
13	2
15	1
12	2
17	1

Number	Occurrence
22	1
11	1
19	1

Mode is 13 and 12 (Bimodal)

Mean, Median, and Mode Cont..

The mode of a set of data is the value in the set that occurs most often.

□ Consider the following list of numbers:

13, 15, 12, 17, 22, 11, 19

Number	Occurrence	Number	Occurrence
13	1	22	1
15	1	11	1
12	1	19	1
17	1		

There is no mode

Handling Missing Values

Field1 (Numeric)	Field2 (Catrgorical)	Field3 (Numeric)	Field4 (Numeric)
21	A	300	67
	A	250	
	B	280	93
24	W		76
22	C	500	85
12		350	66
11		330	
16	A	220	84
16	C		98
17		420	78
18	W	360	89

A set of fields with missing values

Handling Missing Values

Field1 (Numeric)	Field2 (Catrgorical)	Field3 (Numeric)	Field4 (Numeric)
21	A	300	67
	A	250	
	B	280	93
24	W		76
22	C	500	85
12		350	66
11		330	
16	A	220	84
16	C		98
17		420	78
18	W	360	89

A set of fields with missing values

Handling Missing Values (Using Default Values)

Field1 (Numeric)	Field2 (Categorical)	Field3 (Numeric)	Field4 (Numeric)
21	A	300	67
0	A	250	50
0	B	280	93
24	W	240	76
22	C	500	85
12	N	350	66
11	N	330	50
16	A	220	84
16	C	240	98
17	N	420	78
18	W	360	89

Defaults

Field1 Default: 0

Field3 Default: 240

Field2 Default: N

Field4 Default: 50

Handling Missing Values (Using Means and Modes)

- Use the mean for the numeric fields and the mode (if exists) for the categorical fields
- If mode doesn't exist, you need to rely on either a default value or to use a random value
- Numeric Fields: Field1, Field3, and Field4
 - Field1 Mean = $(21+24+22+12+11+16+16+17+18)/9 = 17.44$
 - Field3 Mean = 334.44
 - Field4 mean = 81.78
- If any field doesn't accept decimal values, just approximate the mean value

Handling Missing Values (Using Means and Modes)

- Field2 is categorical, hence we need to compute the mode from the existing values

• Category	Occurrence
• A	3
• B	1
• W	2
• C	2

- Hence, the mode is A

Handling Missing Values (Using Means and Modes)

Field1 (Numeric)	Field2 (Catrgorical)	Field3 (Numeric)	Field4 (Numeric)
21	A	300	67
17	A	250	82
17	B	280	93
24	W	334.44	76
22	C	500	85
12	A	350	66
11	A	330	82
16	A	220	84
16	C	334.44	98
17	A	420	78
18	W	360	89

Assumptions

- Assume Field 1 and Field4 don't accept decimal numbers, Hence we approximate the mean
- Field3 accepts decimal numbers, hence we don't approximate the mean value

Handling Missing Values (Using Random Values)

Field1 (Numeric)	Field2 (Categorical)	Field3 (Numeric)	Field4 (Numeric)
21	A	300	67
22	A	250	73
17	B	280	93
24	W	359.97	76
22	C	500	85
12	C	350	66
11	A	330	86
16	A	220	84
16	C	327.01	98
17	W	420	78
18	W	360	89

Handling Outliers

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	-40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Sample Table

Data Set

Possible Outlier Values

- Outliers are data values that deviate from expected values of the rest of the data set
- Outliers are extreme values that lie near the limits of the data range or go against the trend of the remaining data.
- Normally, outliers need **more investigation** to make sure that they don't occur due to mistakes during data entry

Handling Outliers

Outliers are data points that deviate significantly from the majority of the data in a dataset.

Detecting and dealing with outliers is essential in data preprocessing to ensure that they do not unduly influence analysis or machine learning models.

There are various methods for outlier detection and treatment

Outliers Detection Methods:

Visual Inspection:

- Scatter plots, histograms, and box plots can provide visual cues for identifying outliers.

Z-Score or Standard Score:

- Calculate the z-score for each data point. Data points with z-scores beyond a threshold (e.g., $|z| > 3$) are considered outliers.

IQR (Interquartile Range) Method:

- Calculate the IQR ($Q3 - Q1$) for the data. Data points beyond the thresholds ($Q1 - 1.5 * IQR$) or ($Q3 + 1.5 * IQR$) are considered outliers.

Modified Z-Score:

- Similar to the standard z-score, but it uses the median and median absolute deviation (MAD) instead of the mean and standard deviation.

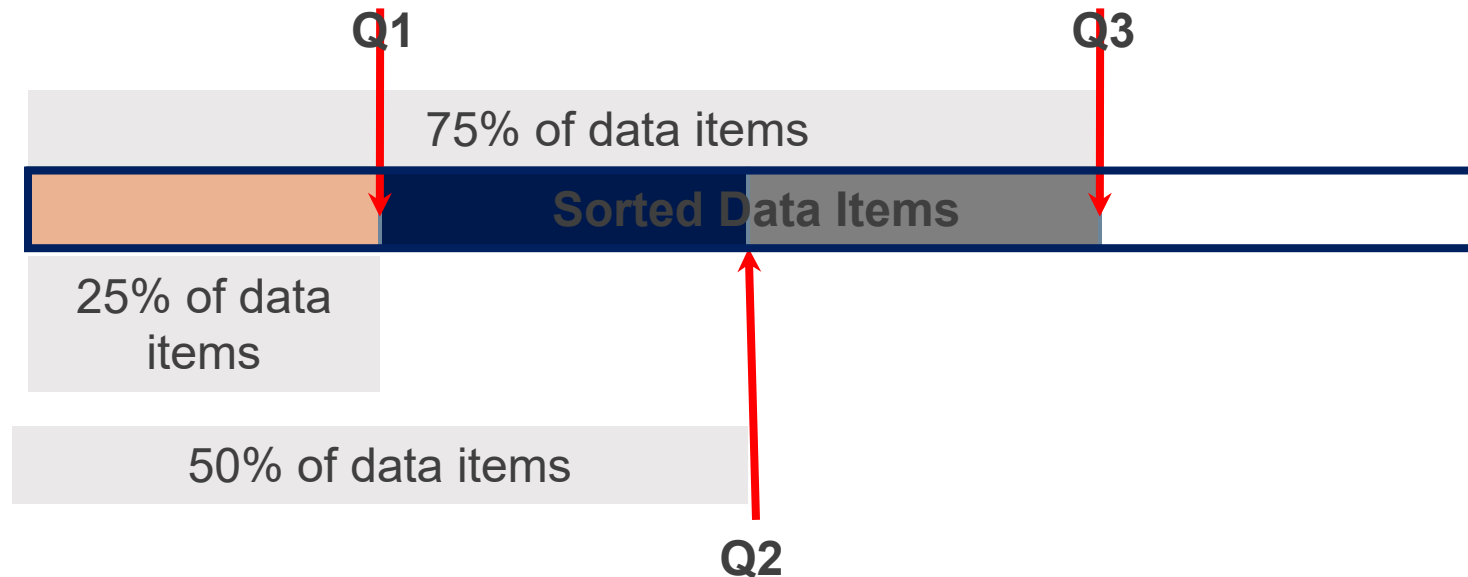
MAD (Median Absolute Deviation):

- Calculate the MAD and use it as a robust measure of spread. Data points beyond a threshold (e.g., $3 * MAD$) are considered outliers.

Local Outlier Factor (LOF):

- Measures the local density deviation of a data point compared to its neighbors to identify local outliers.

Handling Outliers Using Inter-quartile Range



- Quartile is any of the three values which divide the sorted data set into four equal parts
- First quartile (Q1) cuts off lowest 25% of data
- Second quartile (Q2) cuts data set in half (it is the median of the data set)
- Third quartile (Q3) cuts off highest 25% of data, or lowest 75%

Computing Q1, Q2, and Q3

To compute Q1, Q2, and Q3, use the following method:

- ☐ Order the given data set in ascending order.
- ☐ Use the median to divide the ordered data set into two halves. This median is second quartile (Q2). Exclude this median (if it is one of the data items) from any further computation.
- ☐ The first quartile (Q1) value is the median of the lower half of the data.
- ☐ The third quartile (Q3) value is the median of the upper half of the data.

Example #1

- compute Q1, Q2, and Q3 for the following data set:
6, 47, 49, 15, 42, 41, 7, 39, 43, 40, 36

□ Order the given data set in ascending order:

6, 7, 15, 36, 39, 40, 41, 42, 43, 47, 49

□ Q2 = 40 (median of the data set)

Q1 is the median of the lower half of the data (shown in red) with number of median.

□ Q1 = 15

Q3 is the median of the upper half of the data (shown in green) with number of median.

□ Q3 = $(42+43)/2 = 42.5$

Example #2

- compute Q1, Q2, and Q3 for the following data set:

39, 36, 7, 40, 41, 17

- Order the given data set in ascending order:

7, 17, 36, 39, 40, 41

- $Q2 = (36+39)/2 = 37.5$ (median of the data set). Note, the number of data items is even so the median is the average of the middle two data items

- $Q1 = 17$

- $Q3 = 40$

Detecting Outliers using Inter-quartile Range

- Compute the Inter-Quartile Range (IQR) as follows:

$$\text{IQR} = Q3 - Q1$$

- A data value is an outlier if:
its value is $\leq (Q1 - 1.5 \cdot \text{IQR})$, or its value is $\geq (Q3 + 1.5 \cdot \text{IQR})$.

Example of Detecting Outliers using Inter-quartile Range

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	-40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Sample Table

Data Set that might contains outliers

□ Data Set

75000, -40000, 10000000, 50000, 99999

Example Cont..

- Data Set:

75000, -40000, 10000000, 50000, 99999

- Ordered Data Set:

-40000, 50000, 75000, 99999, 10000000

- $Q2 = 75000$

- $Q1 = 50000$

- $Q3 = 99999$

- $IRQ = Q3 - Q1$
 $= 99999 - 50000$
 $= 49999$

- $Q1 - 1.5 * IRQ = 50000 - 1.5 * 49999 = -24998.5$

- $Q3 + 1.5 * IRQ = 99999 + 1.5 * 49999 = 174997.5$

- After delete outliers all data in the data set are within range, hence there is no outliers in this example (Showing in code)

Let's Practices

```
In [1]: import pandas as pd  
import numpy as np
```

```
In [2]: Data = pd.read_csv(r"F:\\Job\\NTI\\Example1.csv")  
Data
```

Out[2]:

	CustomerID	Zip	Gender	Income	Age	MaritalStatus	Transaction Amount
0	1001	10048	M	75000	C	M	5000
1	1002	J2S7K7	F	-40000	40	W	4000
2	1003	90210	NaN	10000000	45	s	7000
3	1004	6269	M	50000	0	S	1000
4	1005	55101	F	99999	30	D	3000

```
In [3]: IN = Data["Income"]  
IN
```

```
Out[3]: 0      75000  
1     -40000  
2    10000000  
3      50000  
4      99999  
Name: Income, dtype: int64
```

```
In [4]: In = np.sort(IN)
In
```

```
Out[4]: array([-40000,  50000,  75000,  99999, 10000000], dtype=int64)
```

```
In [5]: IN.mean()
```

```
Out[5]: 2036999.8
```

```
In [6]: IN.median()
```

```
Out[6]: 75000.0
```

```
In [7]: IN.mode()
```

```
Out[7]: 0      -40000
1       50000
2       75000
3       99999
4    10000000
Name: Income, dtype: int64
```

```
In [8]: Q1 = np.quantile(IN, 0.25)
Q1
```

```
Out[8]: 50000.0
```

```
In [9]: Q3 = np.quantile(IN, 0.75)
Q3
```

```
Out[9]: 99999.0
```

```
In [10]: IQR= Q3 -Q1  
IQR
```

```
Out[10]: 49999.0
```

```
In [11]: lower = Q1 - 1.5*IQR  
lower
```

```
Out[11]: -24998.5
```

```
In [12]: upper = Q3 + 1.5*IQR  
upper
```

```
Out[12]: 174997.5
```

```
In [14]: outliers = IN[((IN<(Q1-1.5*IQR)) | (IN>(Q3+1.5*IQR)))]  
outliers
```

```
Out[14]: 1      -40000  
2      10000000  
Name: Income, dtype: int64
```

```
In [15]: outliers = IN[~((IN<(Q1-1.5*IQR)) | (IN>(Q3+1.5*IQR)))]  
outliers
```

```
Out[15]: 0      75000  
3      50000  
4      99999  
Name: Income, dtype: int64
```

Standard Deviation

Formula

$$\sigma = \sqrt{\frac{\sum (x_i - \mu)^2}{N}}$$

σ = population standard deviation

N = the size of the population

x_i = each value from the population

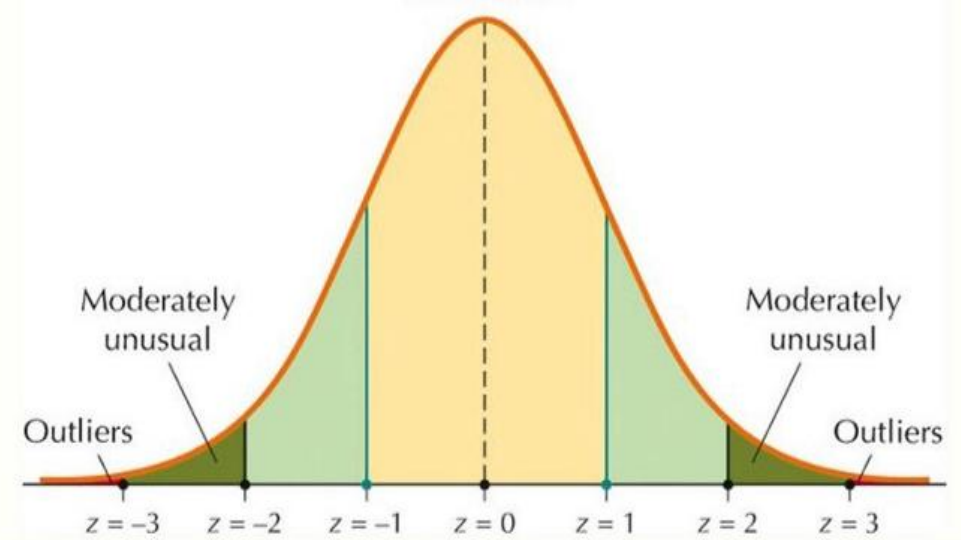
μ = the population mean

To standardize variables, you calculate the mean and standard deviation for a variable. Then, for each observed value of the variable, you subtract the mean and divide by the standard deviation.

A variable that has been rescaled to have a mean of zero and a standard deviation of one.

Detecting Outliers using Modified Z-Score

- The Modified Z-Score method is a robust technique for detecting outliers in a dataset by using the median and the median absolute deviation (MAD) instead of the mean and standard deviation.
- Here's a step-by-step example of how to use the Modified Z-Score method to detect outliers in a dataset:



Example

Step 1: Collect Data

Let's assume you have a dataset of exam scores for a class:

Exam Scores:

78, 85, 92, 70, 65, 88, 95, 60, 150

Step 2: Calculate the Median and MAD:

Median (M): The median is the middle value when the data is sorted. In this case, the sorted scores are:

60, 65, 70, 78, **85**, 88, 92, 95, 150

The median is 85.

Median Absolute Deviation (MAD):

Calculate the absolute deviations of each data point from the median and then take the median of those absolute deviations. The absolute deviations are:

- $|60 - 85| = 25$
- $|65 - 85| = 20$
- $|70 - 85| = 15$
- $|78 - 85| = 7$
- $|85 - 85| = 0$
- $|88 - 85| = 3$
- $|92 - 85| = 7$
- $|95 - 85| = 10$
- $|150 - 85| = 65$

The MAD is the median of these absolute deviations, which is 10.

Step 3: Calculate the Modified Z-Score:

Calculate the Modified Z-Score for each data point using the formula:

$$\text{Modified Z-Score (MZ)} = 0.6745 * (X - M) / \text{MAD}$$

Where:

- **X** is the data point.
- **M** is the median (85 in this case).
- **MAD** is the median absolute deviation (10 in this case).
- **0.6745** is a constant used for consistency with the standard normal distribution.

For each data point:

For 78:

$$MZ = 0.6745 * (78 - 85) / 10 \approx -0.47215$$

For 88:

$$MZ = 0.6745 * (88 - 85) / 10 \approx -0.20235$$

Continue this process for all data points.

Step 4: Identify Outliers

- Data points with a Modified Z-Score greater than a threshold (commonly around 3 or -3) are considered outliers. You can adjust the threshold depending on your specific needs.
- In this example, let's consider a threshold of 3:
- 150 has an MZ of approximately 6.745, which is greater than 3.
- The other data points have MZ values within the range of -3 to 3.
- So, the data point 150 is considered an outlier.

Step 5: Analyze the Detected Outliers

Let's Practices

```
In [17]: df= pd.Series([78, 85, 92, 70, 65, 88, 95, 60, 150])
```

```
In [18]: df.describe()
```

```
Out[18]: count      9.000000  
mean      87.000000  
std       26.565956  
min       60.000000  
25%       70.000000  
50%       85.000000  
75%       92.000000  
max      150.000000  
dtype: float64
```

```
In [19]: #Calculating Median Absolute Deviation  
abs_median = abs(df- df.median())  
abs_median
```

```
Out[19]: 0      7.0  
1      0.0  
2      7.0  
3     15.0  
4     20.0  
5      3.0  
6     10.0  
7     25.0  
8     65.0  
dtype: float64
```

```
In [20]: mad = abs_median.median()  
mad
```

```
Out[20]: 10.0
```

```
In [21]: #Calculating Modified Z-Score  
df = 0.6745 * ((df- df.median()) / mad)  
df
```

```
Out[21]: 0    -0.47215  
1      0.00000  
2      0.47215  
3     -1.01175  
4     -1.34900  
5      0.20235  
6      0.67450  
7     -1.68625  
8      4.38425  
dtype: float64
```

```
In [22]: # Detecting Outlier's from Data  
df[(df > 3.5) | (df < -3.5)]
```

```
Out[22]: 8      4.38425  
dtype: float64
```

```
In [23]: #Removing Outlier's from Data  
df_no_outlier = df[(df < 3.5) & (df > -3.5)]  
df_no_outlier
```

```
Out[23]: 0    -0.47215  
1      0.00000  
2      0.47215  
3     -1.01175  
4     -1.34900  
5      0.20235  
6      0.67450  
7     -1.68625  
dtype: float64
```

Noisy Data

Noisy data are the kind of data that have incorrect values.

Some reasons for noisy data:

- ☐ Data collection instruments may be faulty
- ☐ Human or computer errors may occur during data entry
- ☐ Transmission errors may occur
- ☐ Technology limitations like buffer size, may occur during data-entry

Customer ID	Zip	Gender	Income	Age	Marital Status	Transaction Amount
1001	10048	M	75000	C	M	5000
1002	J2S7K7	F	-40000	40	W	4000
1003	90210		10000000	45	S	7000
1004	6269	M	50000	0	S	1000
1005	55101	F	99999	30	D	3000

Examples of Noisy Data

Smoothing Noisy Data

By smoothing noisy data we can correct the errors.

Smoothing noisy data is performed by:

- ☐ Validation and correction
- ☐ Standardization

Validation and Correction of Noisy Data

- This step examines the data for data-entry errors and tries to correct them automatically as far as possible according to the following guidelines:

- ☐ Spell checking based on dictionary lookup is useful for identifying and correcting misspellings.

Example: **Kairo** can be spell-checked and corrected into **Cairo**

- ☐ Use dictionaries on geographic names and zip codes helps to correct address data.

Example: Zip code **1243456** can be detected as an error since there is no Zip code matches this value

Validation and Correction of Noisy Data (Cont.)

□ Check validation rules and make sure field values follow the rules; for example:

- Age is not less than certain amount and age is a positive number.

Example: if there is a rule governing your data says that age must be between 20 and 60, then ages of 18, 15, and 68 are detected as errors

- Each value of the categorical values belong to certain category.

Example: if all the categories you have are A, B, C, and D, Then if categories W or N are found they will be declared as errors

Validation and Correction of Noisy Data (Cont.)

- ☐ Check the fields that have ambiguous values and check for any possible data-entry errors

Example: Using the same category value to refer to different meaning. “S” in “Marital Status” field could refer to “Single” or “Separated”

Standardization to Smooth Noisy Data

Data values should be consistent and have uniform format.

For example:

- ☐ Date and time entries should have a specific format

Oct. 19, 2009

10/19/2009

19/10/2009

All dates must be written with the same format
that have been agreed upon (e.g., Day/Month/Year)

- ☐ Names and other string data should be converted to either upper or lower case.

MOHAMED AHMED

instead of

Mohamed Ahmed

- ☐ Removing prefixes and suffixes from names.

Mohamed Ahmed instead of Mr. Mohamed Ahmed

Mohamed Ahmed instead of Mohamed Ahmed, Ph.D.

Standardization to Smooth Noisy Data *(Cont.)*

Abbreviations and encoding schemes should consistently be resolved by consulting special dictionaries or applying predefined conversion rules.

US is the standard abbreviation of **United States**

Data Inconsistency

Data inconsistency means that different data items contain discrepancies in their values

It can occur when different data items depend on other data items and their values don't match; for example:

- **Age** and **Birth-date**; age can be computed from the birth-date, hence the value of Age must match the value computed from the birth-date
- **City** and **Phone-area-code**; each city has certain area-code
- **Total-price** and (**unit-price** and **quantity**); total-price can be computed from the unit-price and quantity

These dependencies can be utilized to detect errors and substitute missing values or correct wrong values

Data Inconsistency Example

Client_Code	Place	Phone	Quantity	Unit_Price	Total_Price
101	Cairo	02-99xxxxxx	10	120	1200
102	Cairo	03-99xxxxxx	10	234	2340
102	Alexadria	03-99xxxxxx	20	400	600
104	Port-Said	03-99xxxxxx	30	100	300

Example of Inconsistent Data

Client_Code	Place	Phone	Quantity	Unit_Price	Total_Price
101	Cairo	02-99xxxxxx	10	120	1200
102	Cairo	03-99xxxxxx	10	234	2340
102	Alexadria	03-99xxxxxx	20	400	600
104	Port-Said	03-99xxxxxx	30	100	300

Data Inconsistency Marked in Red

Incorrect Area-Code

Total_Price doesn't equal (Quantity*Unit_Price)

The logo consists of a white hexagon with a black border. Inside the hexagon, the letters "NTI" are written in a large, bold, black sans-serif font. Below "NTI", the words "Data Analysis" are written in a smaller, black, serif font.

NTI

Data Analysis

Data Integration , Transformation and Normalization

Data integration

- Data Integration means that we need to merge data coming from multiple sources.
- This is normally happened in data warehousing and data mining systems.

Issues to be considered during data integration

- **Entity Identification Problem** (we have attributes (coming from different data sources) with different names that represent the same entity (**Student ID , Student no**)
- **Redundancy** (An attribute is redundant if it can be derived from another attribute (**correlation**))
- **Data Value Conflicts** (For the same entity, attribute values from different sources may differ (**meter, kilometer**)
- **Duplication** (there are two or more identical records for a given unique data entry(**discarding all duplication**))

Detecting Redundancies using Correlation Analysis

- Consider two attributes A and B coming from two different data sources. The correlation between attributes A and B can be measured by the correlation coefficient $r_{A,B}$:

$$r_{A,B} = \sum_{i=1}^n (A_i - \bar{A})(B_i - \bar{B}) / \{(n-1)\sigma_A\sigma_B\}$$

Where:

$$\bar{A} = \left(\frac{\sum_{i=1}^n A_i}{n} \right) \quad \bar{B} = \left(\frac{\sum_{i=1}^n B_i}{n} \right) \quad \sigma_A = \sqrt{\frac{\sum_{i=1}^n (A_i - \bar{A})^2}{n-1}} \quad \text{and} \quad \sigma_B = \sqrt{\frac{\sum_{i=1}^n (B_i - \bar{B})^2}{n-1}}$$

- Where n is the number of data items in A or B (assuming they have the same number of data items), $\bar{A}$ and $\bar{B}$ are the mean values of A and B, and σ_A and σ_B are the standard deviations of A and B

Detecting Redundancies using Correlation Analysis (Cont.)

- If the $r_{A,B}$ is greater than 0 then A and B are positively correlated
- If A and B are positively correlated then the values of A increase as the values of B increase
- The higher the value of $r_{A,B}$ the more each attribute implies the other
- So, a high value of $r_{A,B}$ implies that either A or B may be removed as a redundancy but not both of them

Detecting Redundancies using Correlation Analysis (Cont.)

- If the $r_{A,B}$ is 0, then A and B are independent, hence there is no redundancy and we cannot remove any of them
- If $r_{A,B}$ is less than 0 then A and B are negatively correlated
- This implies that the values of one attribute increase as the values of the other decrease and also we cannot remove any of the attributes

Data Normalization

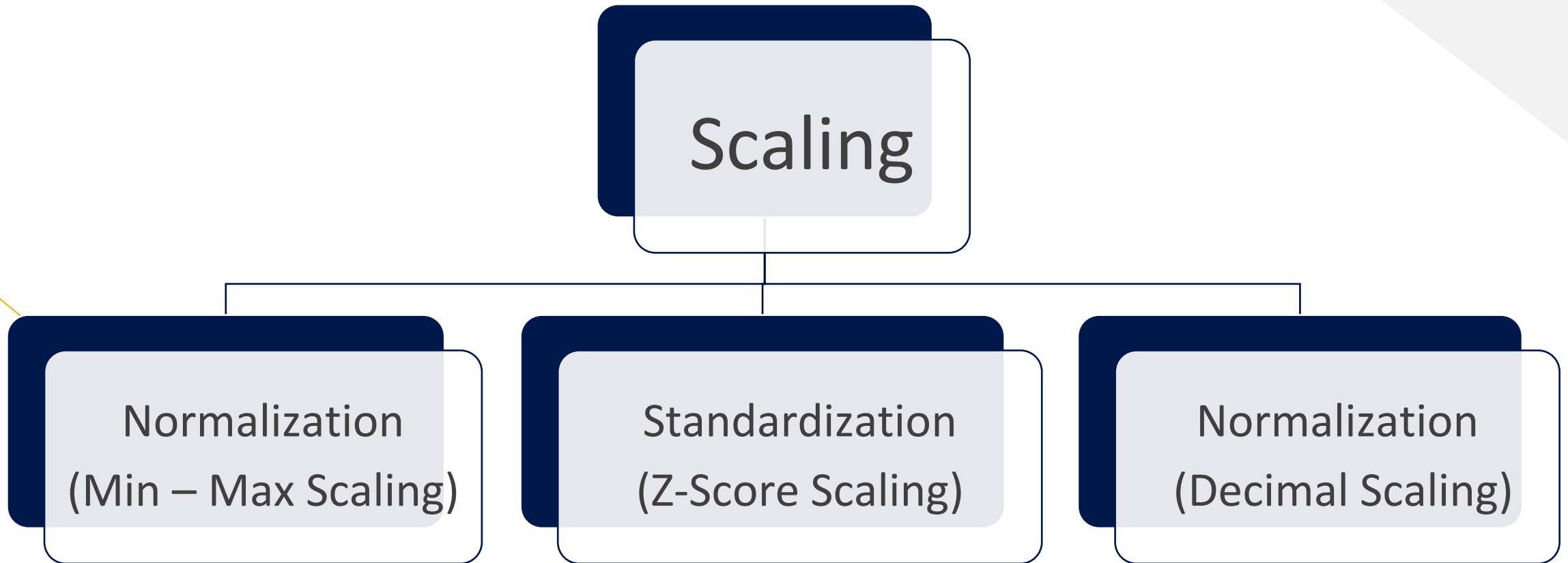
- Data Normalization means that the attribute data are scaled to fall within a small specified range such as -1.0 to $+1.0$, or 0.0 to 1.0
- Data normalization is a critical preprocessing step in data analysis and machine learning to bring data into a standard format or scale, ensuring that different features contribute equally to the analysis and preventing issues caused by varying scales.

Various methods of data normalization

Here are some common types of data normalization:

- Methods of scaling.
 - Min – Max Scaling
 - Z-Score Scaling
 - Decimal Scaling
- Robust Scaling (Scaling by Median and IQR)

Methods of Scaling:



1. Min-Max Normalization:

- Also known as feature scaling, it scales the data between a specified range, typically [0, 1].

Formula: $X_{\text{normalized}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$

- Useful when you have features with varying ranges and want to bring them into a common scale.

1. Min-Max Normalization (Cont.)

- Min-Max normalization performs a linear transformation on the original data
- Suppose that $\min_A$ and $\max_A$ are the minimum and maximum values of an attribute A
- Min-Max normalization maps any value v of A to v' in the range $[\text{new_min}_A, \text{new_max}_A]$ by computing:

$$v' = \left(\left[\frac{(v - \min_A)}{(\max_A - \min_A)} \right] \times [\text{new_max}_A - \text{new_min}_A] \right) + \text{new_min}_A$$

- Min-max normalization preserves the relationships among the original data values

Example:

- Let the minimum and maximum values for the attribute income be \$12000 and \$98000
- To map income to the range [0.0, 1.0], we have:
 $\min_A = 12000$ $\max_A = 98000$
 $\text{new_min}_A = 0.0$ $\text{new_max}_A = 1.0$
- By min-max normalization, a value of \$73600 for income is transformed to:

$$\left(\frac{73600 - 12000}{98000 - 12000} \right) \times [1.0 - 0.0] + 0.0 = 0.716$$

2. Z-Score Normalization

- In Z-Score normalization, the values for an attribute A are normalized based on the mean and standard deviation of A
- A value v of A is normalized to v' by computing:

$$v' = \left(\frac{v - \bar{A}}{\sigma_A} \right) \quad \text{Where: } \bar{A} = \left(\frac{\sum_{i=1}^n A}{n} \right) \text{ and } \sigma_A = \sqrt{\frac{\sum_{i=1}^n (A_i - \bar{A})^2}{n-1}}$$

- Where $\bar{A}$ is the mean value of A and σ_A is the standard deviation of A
- The Z-Score normalization method is useful when the actual minimum and maximum of attribute A are unknown, or when there are outliers that dominate the min-max normalization

Example

- Suppose that the mean and standard deviation of the values for the attribute income are \$54000 and \$16000, respectively
- With Z-Score normalization, a value of \$73600 for income is transformed to

$$\frac{73600 - 54000}{16000} = 1.225$$

3. Normalization by decimal scaling

- This method normalizes by moving the decimal point of values of attribute A
- The number of decimal points moved depends on the maximum absolute value of A
- A value v of A is normalized to v' by computing:

$$v' = \frac{v}{10^j}$$

- Where j is the smallest integer such that $\text{Max}(|v'|) < 1$

Normalization by decimal scaling: Example

- Let the values of A range from -986 to 917
- The maximum absolute value of A is 986
- To normalize by decimal scaling, we divide each value by 1000, which means $j = 3$
- Hence, -986 normalizes to -0.986

Robust Scaling (Scaling by Median and IQR):

- Let Scales data based on the median and the interquartile range (IQR) to minimize the effect of outliers.
- Formula: $X_{\text{robust}} = (X - \text{median}(X)) / \text{IQR}(X)$
- Useful when your data has outliers, and you want a scaling method that is robust to extreme values.

Robust Scaling (Scaling by Median and IQR):

- Robust scaling, also known as scaling by the median and interquartile range (IQR), is a data normalization technique that minimizes the effect of outliers on your data.
- It's particularly useful when your dataset contains extreme values that can significantly affect other scaling methods like standardization (z-score).

Robust Scaling (Scaling by Median and IQR):

- Ages (in years):

18, 21, 22, 25, 29, 31, 34, 35, 42, 170

- Step 1: Collect Data
- Step 2: Calculate the Median and IQR

The median (M) is 29.

Q1: 18, 21, 22, 25, 29 Q1 is 22.

Q3: 31, 34, 35, 42, 170 Q3 is 35.

IQR: $\text{IQR} = \text{Q3} - \text{Q1} = 35 - 22 = 13$

- **Step 3: Perform Robust Scaling:**

Now that you have the median (M) and the IQR, you can scale your data using the robust scaling formula:

$$X_{\text{robust}} = (X - M) / IQR$$

Apply this formula to each data point:

- **For 18 years old:**

$$X_{\text{robust}} = (18 - 30) / 13 \approx -0.9230$$

- **For 21 years old:**

$$X_{\text{robust}} = (21 - 30) / 13 \approx -0.69230$$

Continue this process for all data points, including the outlier (170 years old).

- **Step 4: Analyze the Scaled Data**

Let's Practices in real Dataset

<https://colab.research.google.com/drive/1O1FUYKYVtHPRZWT0lm5Z-Qofv51AKFZk>

Data Cleaning in R

Data Cleaning

Data Cleaning: Is the process to transform raw data into consistent data that can be easily analyzed. It is aimed at filtering the content of statistical statements based on the data as well as their reliability.

Moreover, it influences the statistical statements based on the data and improves your data quality and overall productivity.

Here are a few simple functions you can do to check your data:

1. Duplicates.
2. Data Structure.
3. Finding missing values (NA).
4. Handling Outliers.

```
team points rebounds assists
1      A      4      9      2
2      A      4      9      2
3      B     NA      7     NA
4      C     18      6      7
5      D      6      8      6
6      E     12     NA      6
7      F     14      9      9
8      G      8     14     10
9      H     13     12     NA
10     I     86     11     14
> |
```

Duplicates

```
> duplicated(df)
[1] FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
>
```

```
> sum(duplicated(df))
[1] 1
```

- **To remove duplicate rows in a data frame.**

```
df_without= unique(df)
```

```
df_without
```

```
sum(duplicated(df_without))
```

- **Method selects unique rows**

```
distinct()
```

[dplyr package] to remove duplicate rows in a data frame.

```
install.packages("dplyr")
```

```
library("dplyr")
```

```
dff = distinct(df)
```

```
sum(duplicated(dff))
```

```
> df_without= unique(df)
> df_without
  team points rebounds assists
1    A      4         9        2
3    B     NA         7       NA
4    C     18         6        7
5    D      6         8        6
6    E     12        NA        6
7    F     14         9        9
8    G      8        14       10
9    H     13        12       NA
10   I     86        11       14
> sum(duplicated(df_without))
[1] 0
```

```
> dff = distinct(df)
> dff
  team points rebounds assists
1    A      4         9        2
2    B     NA         7       NA
3    C     18         6        7
4    D      6         8        6
5    E     12        NA        6
6    F     14         9        9
7    G      8        14       10
8    H     13        12       NA
9    I     86        11       14
> sum(duplicated(dff))
[1] 0
```


Data Structure

```
[1] 0
> is.integer(df$points)
[1] TRUE
>
> df$points = as.integer(df$points)
>
> is.integer(df$points)
[1] TRUE
```

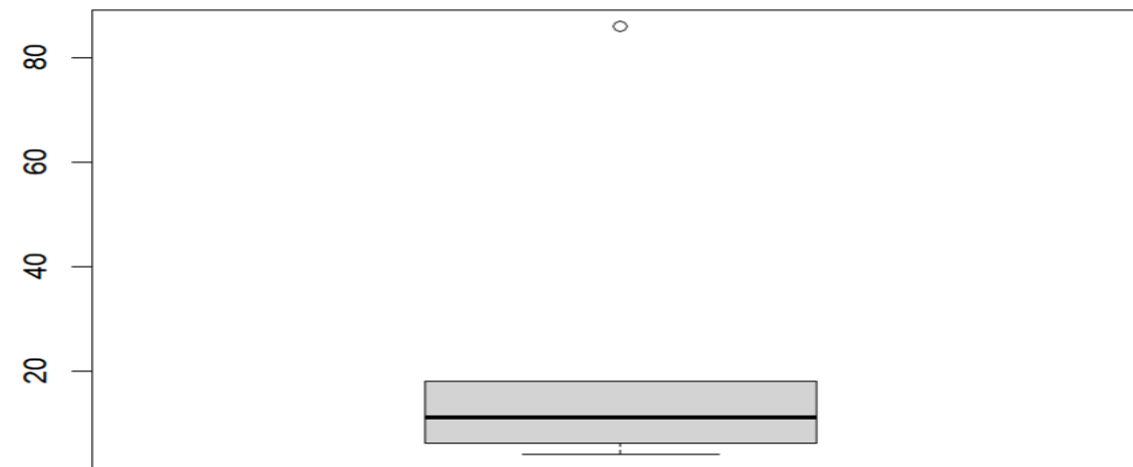
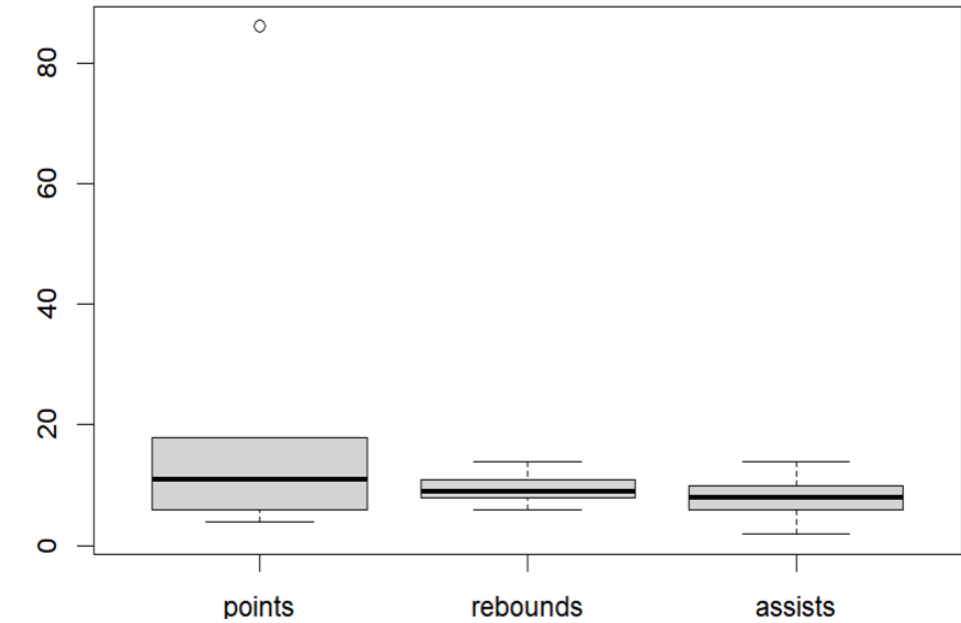
Finding missing values (NA)

```
> is.na(dff)
      team points rebounds assists
[1,] FALSE  FALSE     FALSE   FALSE
[2,] FALSE   TRUE     FALSE    TRUE
[3,] FALSE  FALSE     FALSE   FALSE
[4,] FALSE  FALSE     FALSE   FALSE
[5,] FALSE  FALSE     TRUE    FALSE
[6,] FALSE  FALSE     FALSE   FALSE
[7,] FALSE  FALSE     FALSE   FALSE
[8,] FALSE  FALSE     FALSE    TRUE
[9,] FALSE  FALSE     FALSE   FALSE
> sum(is.na(dff))
[1] 4
```

```
> dff= na.omit(dff)
> dff
  team points rebounds assists
1    A      4        9        2
3    C     18        6        7
4    D      6        8        6
6    F     14        9        9
7    G      8       14       10
9    I     86       11       14
```

Handling Outliers

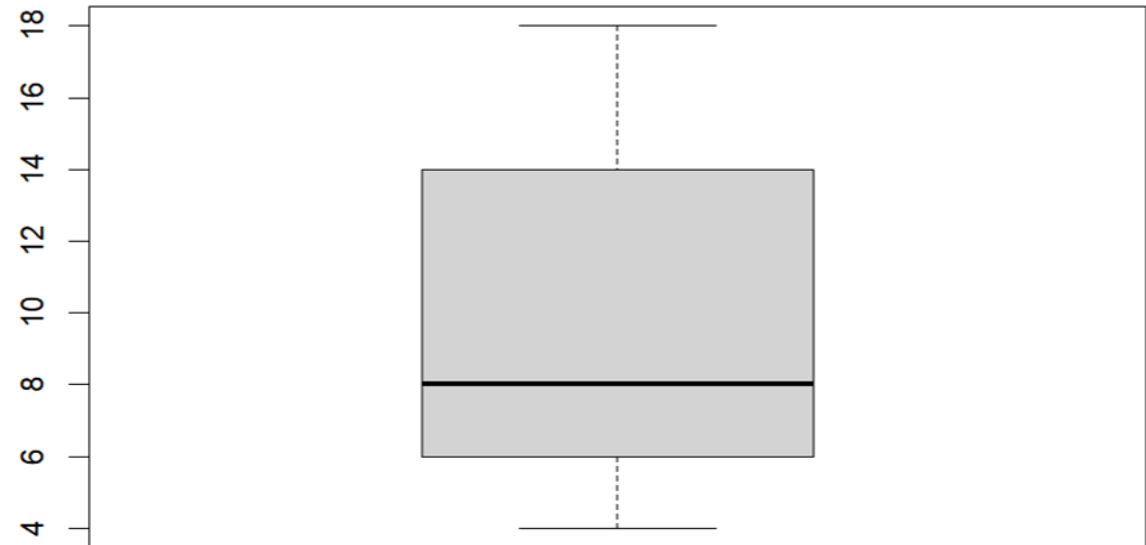
```
> outlier= boxplot(dff$points)$out  
> outlier  
[1] 86
```



```

> # display outliers records
> dff[which(dff$points %in% outlier),]
  team points rebounds assists
9    I     86       11       14
>
> # Remove outliers
> dd = dff[-which(dff$points %in% outlier),]
> dd
  team points rebounds assists
1    A      4         9        2
3    C     18         6        7
4    D      6         8        6
6    F     14         9        9
7    G      8        14       10
/

```





Thank You.



Angelica Astrom



208 555 0164



angelica@fabrikamresidences.com



www.fabrikamresidences.com